x64 系统调用约定与 SSN 提取

在前几章节，我们学会了 APC 和 Early Bird。但无论是 APC 还是 Early Bird，最终都要经过 ntdll.dll 里的 Nt*函数。而 EDR 的用户态钩子，恰恰就插在这些 Nt* 函数的入口处。

本章开始，我们彻底抛弃 ntdll.dll 的调用，自己发 syscall 指令进入内核。这是用户态对抗的终极形态。

- **为什么需要直接系统调用**

➢ **回顾：EDR 的用户态钩子怎么工作的**

EDR 注入个 DLL 到目标进程，在 ntdll.dll!NtWriteVirtualMemory 开头写入条跳转指令：

原始 NtWriteVirtualMemory:

```
4C 8B D1          mov r10, rcx
B8 3C 00 00 00    mov eax, 0x3C    ← SSN
0F 05             syscall
C3                ret
```

被 Hook 后 NtWriteVirtualMemory:

```
E9 XX XX XX XX    jmp EDR 的监控函数   ← 跳转被改写
```

结果：调用 WriteProcessMemory → kernel32 → ntdll!NtWriteVirtualMemory → 跳进 EDR 的监控函数 → EDR 记录你的行为 → 再跳回原始逻辑。

➢ **直接系统调用的核心思路**

既然 EDR 钩子在 ntdll 的函数入口，那就不走 ntdll 的函数入口。我们自己把 syscall 指令写在自己的代码里，直接进入内核。

物理动作：跳过 ntdll.dll 中被 Hook 的那条指令，自己执行 syscall。

- **x64 系统调用的完整流程**

➢ **正常调用链**

```
你的代码：WriteProcessMemory(...)
      ↓
kernel32.dll!WriteProcessMemory（内部包装）
      ↓
ntdll.dll!NtWriteVirtualMemory
      ↓
mov r10, rcx
mov eax, 0x3C          ← SSN（系统服务编号）
syscall                ← 进入内核
      ↓
内核态：KiSystemCall64
      ↓
根据 EAX 中的 SSN，查系统服务表，执行对应的内核函数
```

➢ **x64 调用约定**

x64 Windows 下，前 4 个参数放在寄存器里，第 5 个开始走栈：

参数顺序	寄存器	参数顺序	寄存器
第 1 个	RCX	第 4 个	R9
第 2 个	RDX	第 5 个及以后	栈（从 RSP+0x28 开始）
第 3 个	R8		

关键：syscall 指令本身会把 RCX 覆盖成“返回地址”。

所以内核约定：首个参数不用 RCX 接收，而用 R10 接收。

因此 ntdll 里每个 Nt* 函数的第一条指令都是：

```
mov r10, rcx        ← 把第一个参数从 RCX 备份到 R10
```

- **syscall 指令的物理动作**
  - 把 RCX 保存为“返回地址”（用户态的 RIP 下一条指令）
  - 把 R11 保存为当前 RFLAGS
  - 切换到内核态（Ring 0）
  - 跳转到内核的 KiSystemCall64
  - 内核根据 EAX 里的 SSN，查表找到对应的内核函数
  - 执行完毕后，内核执行 sysret，切回用户态

- **SSN 是什么**

SSN = System Service Number（系统服务编号）。

- 它是每个 Nt* 函数在系统服务表（SSDT）里的索引号。
- 内核收到 syscall 后，看 EAX 里的数字，就知道你要调用哪个函数。

举例（Win11 x64）：

函数	SSN
NtWriteVirtualMemory	0x3C
NtAllocateVirtualMemory	0x18
NtProtectVirtualMemory	0x50
NtCreateThreadEx	0xC1

注意：SSN 是按 Windows 版本变化的。Win10 和 Win11 的同个函数，SSN 可能不同。所以不能硬编码，必须动态提取。

- **从 ntdll.dll 机器码提取 SSN**

- **Nt* 函数的典型机器码**

在 x64 下，ntdll.dll 里的 Nt* 函数开头通常是：

```
4C 8B D1                mov r10, rcx
B8 XX XX XX XX          mov eax, SSN                ← 这 4 个字节就是 SSN
F6 04 25 08 03 FE 7F    test byte ptr [7FFE0308h], 1
75 03                   jne +3
0F 05                   syscall
C3                      ret
```

关键：B8 是 mov eax, imm32 的操作码。B8 后紧跟的 4 个字节就是 SSN（小端序）。

- **提取逻辑**

```
拿到 ntdll.dll 基址
拿到 NtWriteVirtualMemory 的函数地址
检查函数开头是不是 4C 8B D1 B8
读取 B8 后面的 4 字节（偏移 +4 处）
这就是 SSN
```

- **代码示例（见文件夹）**

➢ **第一版：明文 API 实现**

运行结果（Windows 11 示例）：

```
[*] NtWriteVirtualMemory 地址: 0x7ffa12345678
[*] 前 8 字节: 4c 8b d1 b8 3c 00 00 00
[+] NtWriteVirtualMemory SSN: 0x3c (60)
```

如果前 8 字节不是 4c 8b d1 b8，说明：

- EDR 已经 Hook 了这个函数（开头变成了 E9 跳转）。
- 或 Windows 版本不同，存根格式变了。

➢ **第二版：无 IAT 版本 + 完整 syscall 存根**

这版用我们自己的 GetProcAddressEx 解析 ntdll 和函数地址，并且真正用 syscall 调用一次 NtWriteVirtualMemory。

关键点：

DoSyscall 的汇编：

- EAX = SSN
- RCX/RDX/R8/R9 分别放 4 个参数
- R10 = RCX（内核约定，必须做）
- syscall
- EAX 里是返回值

参数 5 及以后：

- 如果要传第 5 个参数，必须在 syscall 前把它压到栈上 RSP+0x28 处，并保证 RSP 16 字节对齐。

GCC vs MSVC：

- GCC（MinGW）支持 __asm__ 内联汇编，上面的代码可以直接编译。
- MSVC x64 不支持 __asm，需要用 .asm 文件 + ml64 编译，或者用 intrin.h 里的 __syscall 之类的封装（不推荐）。

- **MSVC 用户的补充说明**

如果用 MSVC，需要写个单独的 .asm 文件：

```
; syscall_stub.asm
.code
DoSyscall PROC
    mov r10, rcx
    mov eax, ecx        ; 第一个参数是 SSN，但注意这里参数顺序要调整
```

```
    ; ... 实际需要按参数顺序重新组织
    syscall
    ret
DoSyscall ENDP
END
```

或用更简单的方案：用函数指针+硬编码机器码，把 syscall 存根的机器码写进个可执行内存块后调用。这是很多红队框架的做法（比如 Cobalt Strike 的 syscall 实现）。